Architecting and Designing Software

Adapted from:

Timothy Lethbridge and Robert Laganiere,
Object-Oriented Software Engineering –
Practical Software Development using UML and Java, 2005
(chapter 9)

9.1 The Process of Design

Definition:

- *Design* is a problem-solving process whose objective is to find and describe a way:
  - —To implement the system's *functional requirements*...
  - —While respecting the constraints imposed by the *non-functional requirements*...
    - including the budget
  - —And while adhering to general principles of *good quality*

Design as a series of decisions

A designer is faced with a series of *design issues*.

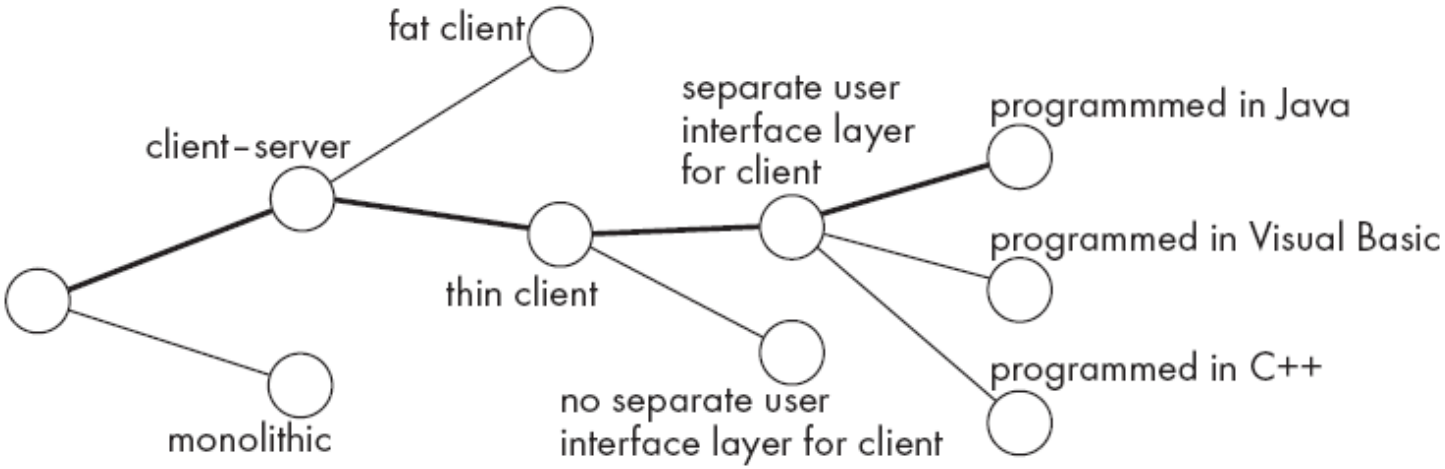
- These are sub-problems of the overall design problem.
- Each issue normally has several alternative solutions, also known as design *options*.
- The designer makes a *design decision* to resolve each issue.
  - —This process involves choosing the best option from among the alternatives.

To make each design decision, the software engineer uses:

- knowledge of the requirements;
- knowledge of the design as created so far;
- knowledge of the technology available;
- knowledge of software design principles and ‘best practices’;
- knowledge of what has worked well in the past.

Design space

- Two designers rarely come up with exactly the same solution.
- The space of possible designs that could be achieved by choosing different sets of alternatives is often called the *design space*.
- The figure illustrates this concept:



- Each design decision should be recorded, along with the *reasoning* that went into making the decision (known as a *design rationale*).

Subsystems, components and modules

- **A *module* can be seen as the basic building block which is used in modularisation of software systems. In general, a module is *not* an independent system.**
  - A *module* is an element that is defined at the programming language level.
    - Examples of modules:
      - methods, classes and packages in Java, or
      - Functions and file modules in C.
- **A *component* is a physical and replaceable part of a system that conforms to and provides the realization of a set of interfaces [BRJ99].**
- **A *system* is an ensemble of related elements that form a unified whole. A software engineering system accomplishes a set of definable responsibilities or objectives.**
- **A *subsystem* is a system in its own right:**
  - It is part of a larger system
  - It has a definite interface

Top-down and bottom-up design

Top-down design

- First design the very high level structure of the system.
- Then gradually work down to detailed decisions about low-level constructs.
- Finally arrive at detailed decisions such as:
  - —the format of particular data items;
  - —the individual algorithms that will be used.

Bottom-up design

- Make decisions about reusable low-level utilities.
- Then decide how these will be put together to create high-level constructs.

A mix of top-down and bottom-up approaches is normally used:

- Top-down design is almost always needed to give the system a good structure.
- Bottom-up design is normally useful so that reusable components can be created.

9.2 Principles Leading to Good Design

Some overall goals we want to achieve when doing good design are:

- Increasing profit by reducing cost and increasing revenue
- Ensuring that we actually conform with the requirements
- Accelerating development
- Increasing qualities such as
  - —Reliability
  - —Maintainability
  - —Reusability
  - —Efficiency

Design Principle 1: Divide and conquer

Trying to deal with something big all at once is normally much harder than dealing with a series of smaller things

- Separate people can work on each part.
- An individual software engineer can specialize.
- Each individual component is smaller, and therefore easier to understand.
- Parts can be replaced or changed without having to replace or extensively change other parts.

A software system can be divided in many ways:

- A distributed system is divided up into clients and servers
- A system is divided up into subsystems
- A subsystem can be divided up into one or more packages
- A package is divided up into classes
- A class is divided up into methods

Design Principle 2: Increase cohesion where possible

A subsystem or module has high cohesion if it keeps together things that are related to each other, and keeps out other things.

- Cohesive modules make the system as a whole easier to understand and change.
- The different types of cohesion (ordered from highest to lowest in terms of the precedence you should normally give them when making design decisions):
  - —Functional
  - —Layer
  - —Communicational
  - —Sequential
  - —Procedural
  - —Temporal
  - —Utility

Functional cohesion

This is achieved when a module only performs a single computation, and returns a result, without having side effects.

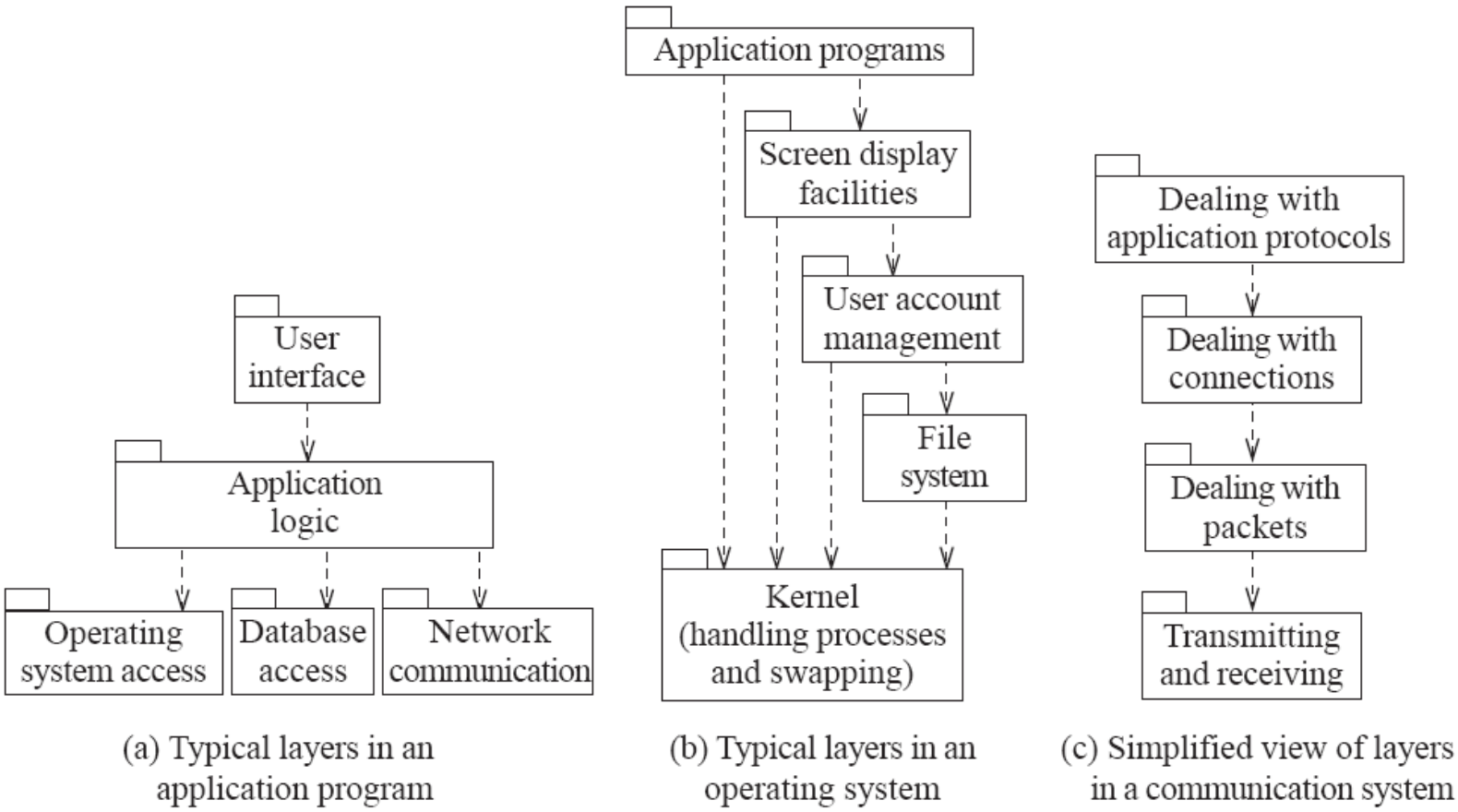
- A module lacks side effects if performing the computation leaves the system in the same state it was in before performing the computation.
  - —Modules that update a database, interact with the user, or create a new file are *not* functionally cohesive.
- Benefits to the system:
  - —Easier to understand
  - —More reusable
  - —Easier to replace

Layer cohesion

This is achieved when the facilities for providing a set of related *services* to the user or to higher-level layers are kept together, and everything else is kept out.

- The layers should form a hierarchy
  - —Higher layers can access services of lower layers
  - —Lower layers do not access higher layers
- Side effects are allowed, and are often essential (for example in a database access layer).
- The set of procedures through which a layer provides its services is commonly called the *application programming interface (API)*
- You can replace a layer without having any impact on the other layers
  - —You just replicate the API

Examples of the use of layers



(a) Typical layers in an application program

(b) Typical layers in an operating system

(c) Simplified view of layers in a communication system

Communicational cohesion

This is achieved when all modules (processing elements) that access or manipulate certain data are kept together (e.g. in the same class), and everything else is kept out.

- One of the strong points of the object-oriented paradigm is that it helps ensure communicational cohesion.
- Main advantage of communicational cohesion: when you need to make changes to the data, you find all the code in one place.
- You should not sacrifice layer cohesion to achieve communicational cohesion:
  - —For example, even though objects may be stored in a database or on a remote host, a class must only load and save (data) objects using the services in the API of lower layers.

Sequential cohesion

This is achieved when a series of procedures, in which one procedure provides input to the next, are kept together – and everything else is kept out.

- You should not sacrifice communicational cohesion to achieve sequential cohesion.
  - —Methods in different classes might provide inputs to each other and be called in sequence, but they would each be kept in its own class.

Procedural cohesion

This is achieved when you keep together several procedures that are executed one after another, even if one does not necessarily provide input to the next.

- Weaker than sequential cohesion.

Temporal Cohesion

This is achieved when operations that are performed during the same phase of the execution of the program are kept together, and everything else is kept out.

- For example, placing together the code used during system start-up or initialisation.
- Weaker than procedural cohesion.

Utility cohesion

This is achieved when utilities which are logically related are kept together.

- A *utility* is a procedure or class that has wide applicability to many different subsystems and is designed to be reusable.
- For example, the **`java.lang.Math`** class has utility cohesion.
  - —Remark that the functions in **`java.lang.Math`** do not exhibit communicational, sequential, procedural or temporal cohesion. They are only logically related.

Design Principle 3: Reduce coupling where possible

***Coupling* occurs when there are *interdependencies* between one module and another**

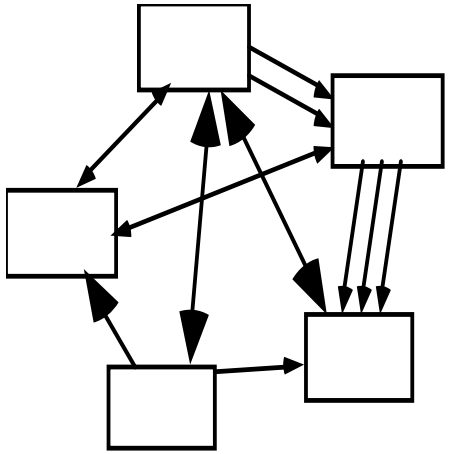
- When interdependencies exist, changes in one place will require changes somewhere else.
- A network of interdependencies makes it hard to see at a glance how some component works.

Reduce coupling where possible

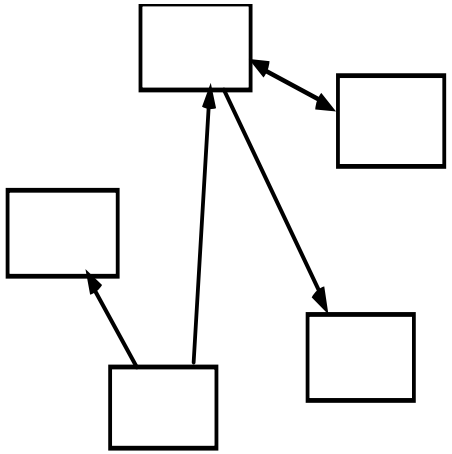
•Types of coupling (the types at the top are the strongest, and hence the most important to avoid):

—Content
—Common
—Control
—Stamp
—Data
—Routine Call
—Type use
—Import
—External

A tightly coupled system



A loosely coupled system



- To reduce coupling you have to reduce the *number* of connections between modules and the *strength* (suggested by the boldness of the arrows in the figure above) of the connections.

Content coupling

This occurs when a module *surreptitiously* modifies data that is *internal* to another module.

- Content coupling should always be avoided, since any modification of data should be easy to find and easy to understand.
- In order to reduce content coupling you should *encapsulate* all instance variables
  - —declare them `private`
  - —and provide get and set methods
- A more subtle form of content coupling occurs when you directly modify an instance variable *of* an instance variable (even if all instance variables are private), as in the following example:

Example of content coupling

```
public class Line
{
  private Point start, end;
  ...
  public Point getStart() { return start; }
  public Point getEnd()  { return end; }
}

public class Arch
{
  private Line baseline;
  ...
  void slant(int newY)
  // Modifies the y value of the end Point of baseline
  {
    Point theEnd = baseline.getEnd();
    theEnd.setLocation(theEnd.getX(),newY);
  }
}
```

Common coupling

This occurs whenever you use a global variable – all the modules using the global variable become coupled to each other, and to the module that declares the variable.

- In Java, public static variables serve as global variables.
- A weaker form of common coupling is when a variable can be accessed by a *subset* of the system's classes
  - —e.g. a Java package
- Common coupling can be acceptable for creating global variables (or constants) that represent system-wide default values.
  - —It would be more complex to force a large number of routines to pass around such information as their parameters.

Control coupling

This occurs when one procedure calls another using a 'flag' or a 'command' that explicitly controls what the second procedure does.

—The following is an example:

```
public routineX(String command)
{
  if (command.equals("drawCircle")
  {
    drawCircle();
  }
  else
  {
    drawRectangle();
  }
}
```

- To make a change you have to change both the calling and the called method.
- Where possible, the use of polymorphic operations represent the best way to reduce control coupling.

Stamp coupling

This occurs whenever one of your application classes is declared as the *type* of a method argument.

- Since one class now uses the other (there is a semantic dependency), changing the system becomes harder
  - —Reusing one class requires reusing the other

- Two ways to reduce stamp coupling:
  - —using an interface as the argument type
  - —passing simple variables

Example of stamp coupling

```
public class Emailer
{
  public void sendEmail(Employee e, String text)  {...}
  ...
}
```

• The problem is that sendMail() *does not need* to be given access to the full Employee object. It may only need to access the name and the email of an Employee.

Two ways to reduce stamp coupling

Using an interface as the argument type:

```
public interface Addressee {
  public abstract String getName();
  public abstract String getEmail();
}

public class Employee implements Addressee   {…}

public class Emailer {
  public void sendEmail(Addressee e, String text)   {...}
  ...
}
```

Using simple data types (this replaces stamp coupling with data coupling):

```
public class Emailer {
  public void sendEmail(String name, String email, String text)   {...}
  ...
}
```

Data coupling

This occurs whenever the types of method arguments are either primitive or else simple library classes.

- The more arguments a method has, the higher the coupling
  —All methods that use the method must pass all the arguments
- You should reduce data coupling by not giving methods unnecessary arguments.
- Methods must obviously have arguments, therefore some data or stamp coupling is unavoidable.

Routine call coupling

This occurs when one routine (or method in an object oriented system) calls another.

- The routines are coupled because they depend on each other's behaviour.
- Routine call coupling is always present in any system.
- If you repetitively use a sequence of two or more methods to compute something then you can reduce routine call coupling by writing a single routine that encapsulates the sequence.

Type use coupling

This occurs when a module uses a data type defined in another module.

- It occurs any time a class declares an instance variable or a local variable as having another class for its type.
- The consequence of type use coupling is that if the type definition changes, then the users of the type may have to change.
- To reduce type use coupling always declare the type of a variable to be the most general possible class or interface that contains the required operations.

Import coupling

This occurs when one module imports another module (e.g. a package in Java).

- The importing module is now exposed to everything in the imported module (the importing module *depends* on the imported module).
- If the imported module changes something or adds something, this may raise a conflict with something in the importing module, forcing the importing module to change.
  - —Some import coupling is necessary, since it enables you to use the libraries of the system.
  - —It is important not to import packages or classes that you do not need.

External coupling

This occurs when a module has a dependency on the operating system, a particular type of hardware, some software written by a third party, etc.

- It corresponds to a relatively high level of coupling (lower than common coupling, but higher than control coupling, according to [Pre97]).
- It is best to reduce the number of places in the code where such dependencies exist.
- The Façade design pattern can reduce external coupling by providing a very small interface to external facilities.

Design Principle 4: Keep the level of abstraction as high as possible

You should ensure that your designs are abstract, i.e. that your designs allow you to hide or defer consideration of details, thus reducing complexity

- Abstractions allow you to understand the essence of a (sub)system without having to know unnecessary details.
- Abstractions are needed because the human brain can process only a limited amount of information at any one time.
  - —You can use the UML to model a software system at different levels of abstraction, by presenting diagrams with different levels of details [BRJ99].
  - —Procedural and data abstractions are supported directly at the programming language level.

Design Principle 5: Increase reusability where possible

You should design the various aspects of your system so that they can be used again in other contexts, both in your system and in other systems.

- You can build reusability into procedures, classes or frameworks
  - —A framework is an extensible, domain-specific, architectural pattern [BRJ99, JBR99].
- Basic strategies for increasing reusability:
  - —Generalize your design as much as possible
  - —Follow the preceding three design principles (increase cohesion, reduce coupling, and increase abstraction)
  - —Simplify your design as much as possible

Design Principle 6: Reuse existing designs and code where possible

Design with reuse is complementary to design for reusability.

- Actively reusing designs or code allows you to take advantage of the investment you or others have made in reusable components
  - —*Code cloning* (i.e. copying code from one place to another) should not be seen as a form of reuse

Design Principle 7: Design for flexibility

Design for *flexibility* (also known as *adaptability*) means actively anticipating possible future design changes, and prepare for them.

- Ways to build flexibility into your system:
  - —Reduce coupling and increase cohesion
  - —Create abstractions
  - —Do not hard-code anything
  - —Leave all design options open
    - Do not restrict the options of people who have to modify the system later
  - —Use reusable code and make code reusable

Design Principle 8: Anticipate obsolescence

Anticipating obsolescence means planning for evolution of the technology or environment so that the software will continue to run or can be easily changed.

- To better anticipate obsolescence:
  - —Avoid using early releases of technology
  - —Avoid using software libraries that are specific to particular environments
  - —Avoid using undocumented features or little-used features of software libraries
  - —Avoid using software or special hardware from companies that are less likely to provide long-term support
  - —Use standard languages and technologies that are supported by multiple vendors

Design Principle 9: Design for Portability

Design for portability has as objective the ability to have the software run on as many platforms as possible.

- To achieve portability avoid the use of facilities that are specific to one particular environment.
    - E.g. a library only available in Microsoft Windows.

Design Principle 10: Design for Testability

During design you can take steps to make testing easier.

- It pays off to design a system so that *automatic testing* is made easy.
  - —The most important way to design for testability is to ensure that all the functionality of the code can be executed without going through a graphical user interface.
    - You can achieve this by carefully separating the UI from the functional layer of the system.
      - » Another good strategy is to provide a command-line version of your system.
    - A test harness can then be written that calls the API of the functional layer.

Design for testability

- Methods can be written within each class to act as test drivers for the methods of that class.
    - In Java, you can create a main() method (to act as a test driver) in each class in order to exercise the other methods.

—In order to ensure that the behaviour of a sub-class is consistent with the behaviour of its super-class, the driver methods of the super-class must be run in each sub-class.

Design Principle 11: Design defensively

You should never trust how others will try to use a component you are designing.

- Handle all cases where other code might attempt to use your component inappropriately
- Check that all of the inputs to your component are valid. In practice, you should check the *preconditions* of each component.
  - —Unfortunately, over-zealous defensive design can result in unnecessarily repetitive checking

Design by contract

Design by contract is a technique that allows you to design defensively in an efficient and systematic way.

- Key idea
  - —each method has an explicit *contract* with its callers.
- The contract has a set of assertions that state:
  - —What *preconditions* the called method requires to be true when it starts executing
  - —What *postconditions* the called method agrees to ensure are true when it finishes executing
  - —What *invariants* the called method agrees will not change as it executes

9.5 Software Architecture

Software architecture **refers to the overall structure of the software system and the way in which that structure provides semantic integrity for the system [SG96].**

- The architectural design process is concerned with:
  - —Dividing software into subsystems
  - —Determining their interfaces
  - —Deciding how the subsystems will interact
- The documentation produced as a result of the architectural design process is often called the *architectural model*.
- The architecture is the core of the design, so all software engineers need to understand it.
- The architecture will often constrain the overall efficiency, reusability and maintainability of the system.
  - —Poor decisions made while creating the architectural model will constrain subsequent design.

The importance of software architecture

There are four main reasons why you need to develop an architectural model:

- To enable everyone to better understand the system
- To organize development and to allow people to work on individual subsystems in isolation
- To prepare for extension of the system
- To facilitate reuse and reusability

Contents of a good architectural model

A system's architecture will often be expressed in terms of several different *views*

- The logical breakdown into subsystems and the interfaces among the subsystems
- The dynamics of the interaction among subsystems and components at run time
- The data that will be shared among the subsystems
- The components that will exist at run time, and the machines or devices on which they will be located

Remarks:

—An architecture description looks like a complete system description, but it is smaller: it only contains the architecturally relevant models and views of the system [JBR99].

—An architectural model comprises both structural and behavioral aspects of the system.

Design stable architecture

To ensure the maintainability and reliability of a system, an architectural model must be designed to be *stable*.

- Being stable means that the new features can be easily added with only small changes to the architecture
- In the Unified Process (UP) [JBR99], by definition, at the end of the *elaboration* phase the software architecture should be stable.
  - —The success of a UP project depends on the ability of a small (but well-trained) team of architects and developers to produce a stable architecture without spending a large amount of resources
    - According to [JBR99], it is possible to develop a stable architecture when only approx. 30% of the first product release has been invested.

Developing an architectural model

- **First, you build a tentative architecture (before considering the detailed use cases). At this stage you should use:**
  - —Good understanding of the *domain area*,
  - —General architectural patterns (discussed next).
  - —*Suggestion*: have several different teams independently develop a first draft of the architecture and merge together the best ideas
- **Next, you pick and implement a set of use cases and build an even better architecture, and so on.**
  - —In each iteration
    - you further develop the application-specific parts of the architecture, which may include any architecturally relevant aspects of the system.
    - consider each use case and adjust the architecture to make it realizable
    - mature the architecture
  - —You should start with the *architecturally relevant use cases*.

Developing an architectural model

The architecturally relevant use cases are the ones that [JBR99]:

- Help you to mitigate the most serious risks
- Are most important to the user of the system

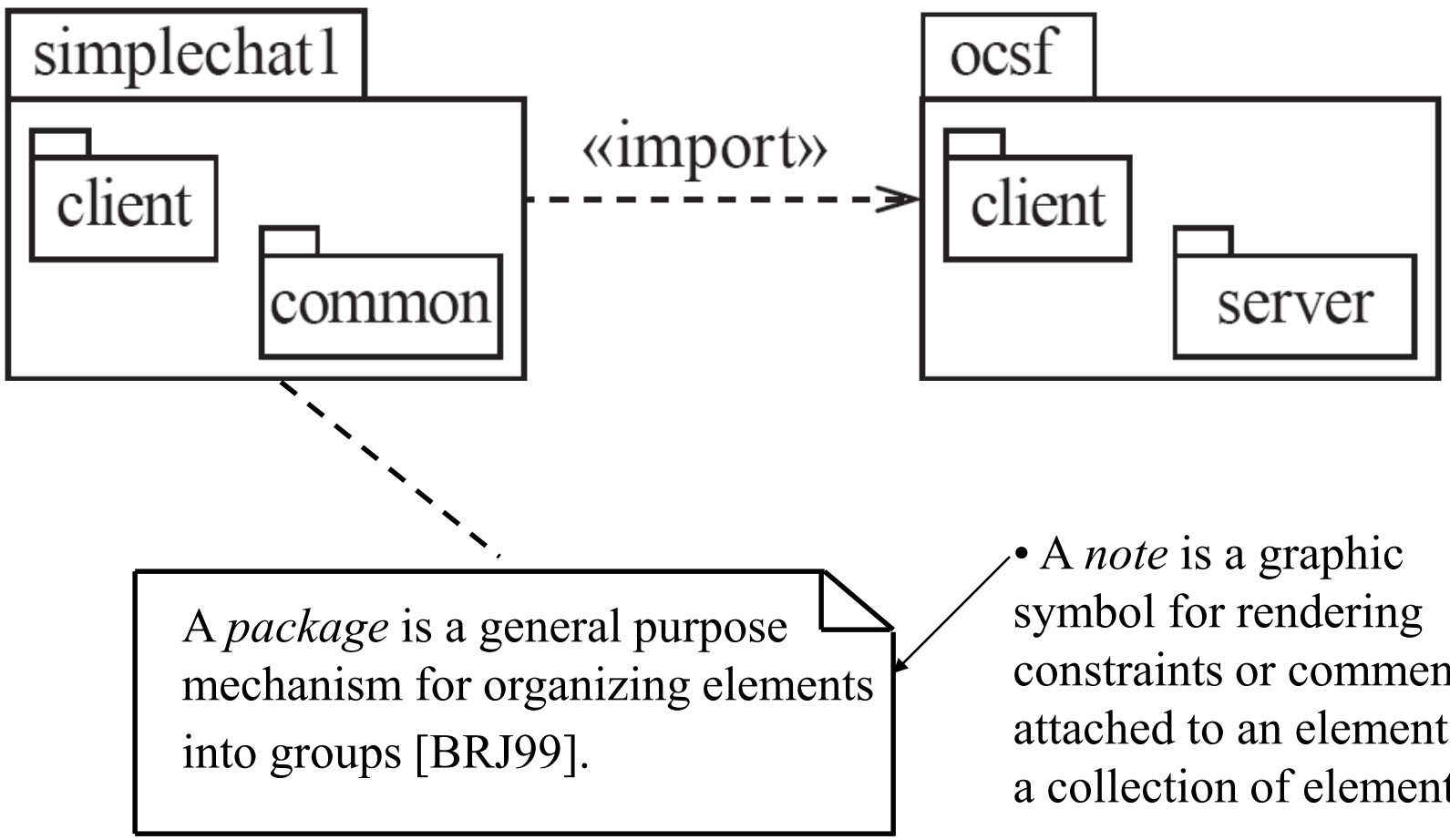
  —Example

  - Consider an ATM (Automated Teller Machine) system that implements four use cases: Withdraw Money, Deposit Money, Transfer between Accounts, and View Balance.
  - The software architect decides that the single architecturally relevant use case is Withdraw Money [JBR99].
- Help you to cover all important functionality, so that nothing remains in shadow.

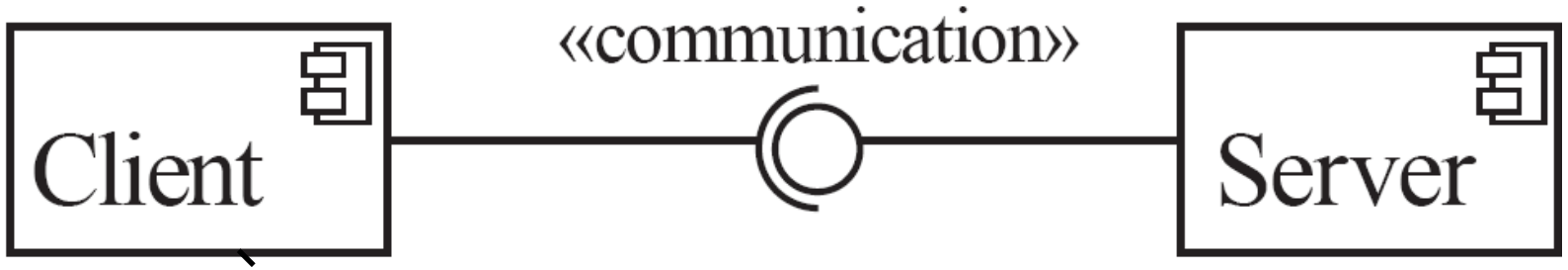
Describing an architecture using UML

- All UML diagrams can be useful to describe aspects of the architectural model
- The following UML diagrams are particularly important for architecture modelling:
  - —Package diagrams
  - —Component diagrams
  - —Deployment diagrams

Package diagrams



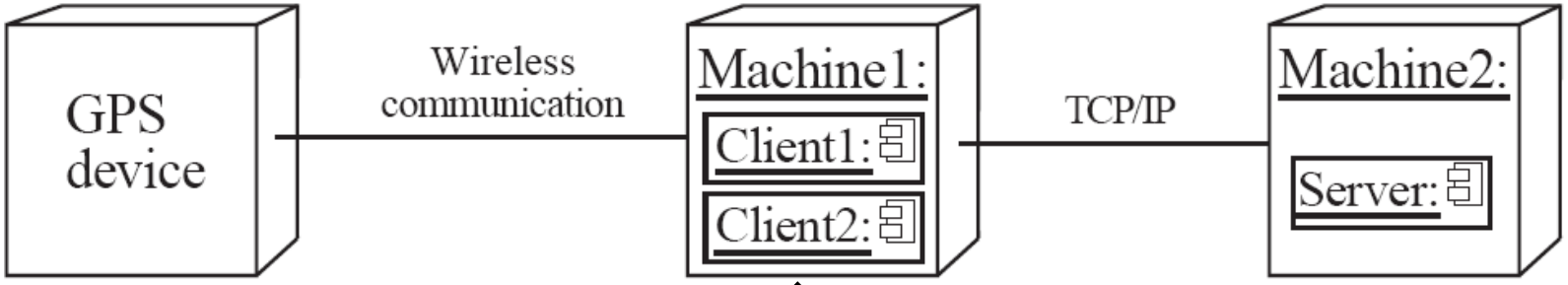
Component diagrams



A *component* is a physical and replaceable part of a system that conforms to and provides the realization of a set of interfaces [BRJ99].

- A component *provides* one or more interfaces for other components to use.
- To show a component *using* an interface provided by another you use a semi-circle at the end of a line.

Deployment diagrams



Each node of a deployment diagram can include one or several run-time software components.

A *node* is a physical element that exists at run time and represents a computational resource, generally having at least some memory, and, often, processing capability [BRJ99].

9.6 Architectural Patterns

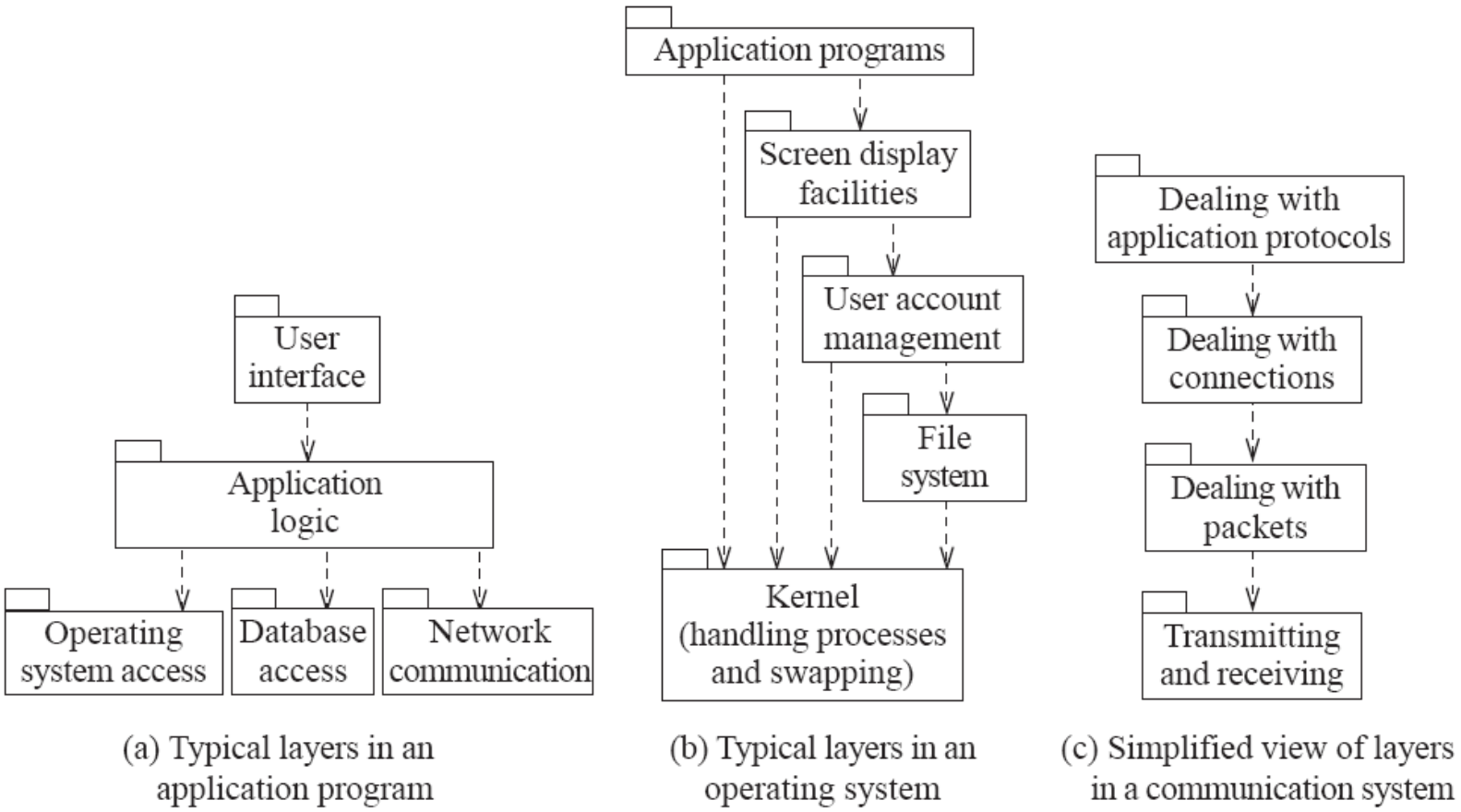
- **The notion of patterns can be applied to software architecture.**
  - *Architectural patterns* (also called *architectural styles*) allow you to design software systems using subsystems and components that are as independent of each other as possible.
  - A *pattern* is the outline of a reusable solution to a general problem encountered in a particular context. Patterns are used at different levels of abstraction:
    - —*Design patterns* - such as Observer, Façade or Proxy – are widely used in software modelling and design
    - —*Architectural patterns* – such as Layers, Client-Server or Pipe-and-Filter - represent standard solutions to commonly occurring architectural problems

The Multi-Layer architectural pattern

In a layered system, each layer communicates only with the layers below it – in many cases, only the layer immediately below it.

- Each layer has a well-defined interface (the API defining the services it provides and) used by the layer (immediately) above.
  - —A higher layer sees a lower layer as a set of *services*.
  - —Lower layers are not aware of any details or interfaces in the upper layers.
- A complex system can be built by superimposing layers at increasing levels of abstraction.
  - —For example, in an application program
    - It is important to have a separate layer for the UI (independence of the UI layer allows the application to use several different UIs)
    - Layers immediately below the UI layer provide the application functions determined by the use-cases
    - Bottom layers provide general services, such as network communication and database access

Examples of multi-layer systems



(a) Typical layers in an application program

(b) Typical layers in an operating system

(c) Simplified view of layers in a communication system

The Client-Server and other distributed architectural patterns

- **There is at least one component that has the role of *server*, waiting for and then handling (or serving) connections.**
- **There is at least one component that has the role of *client*, initiating connections in order to obtain some service.**

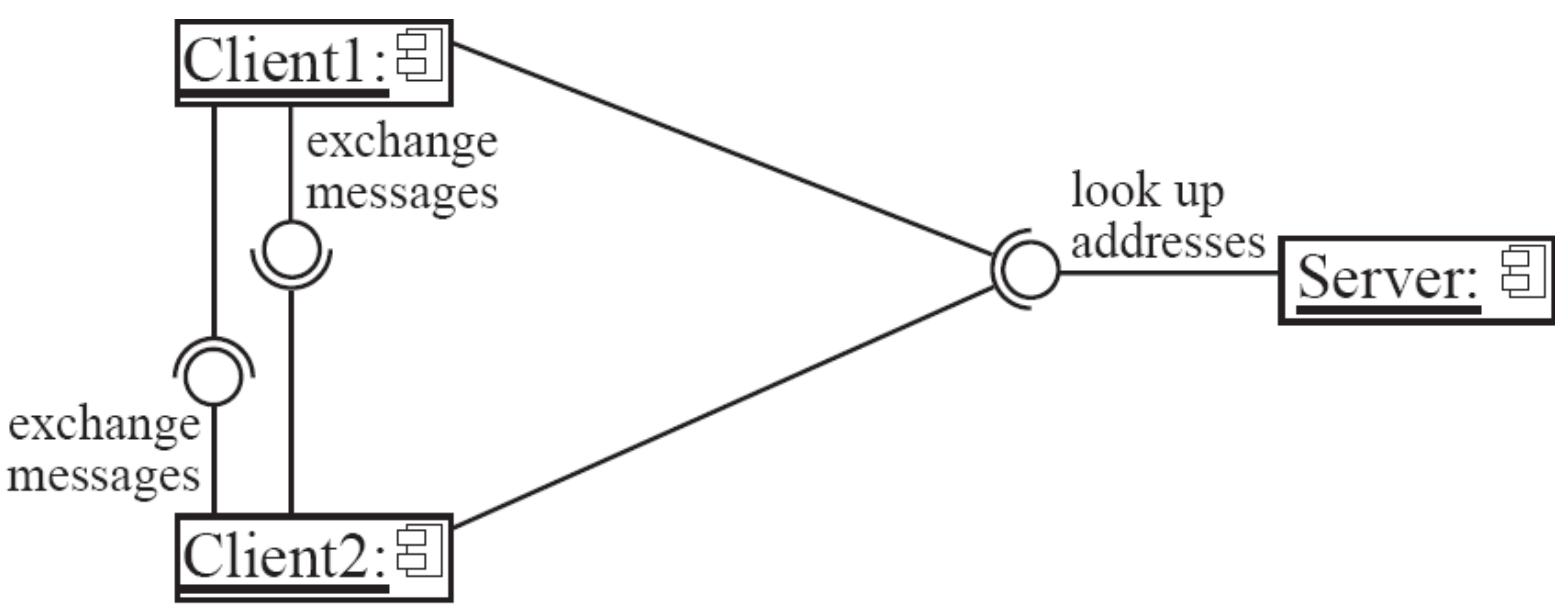
—An important variant of the (above two-tier) client-server architecture is the **three-tier** model, which consists of three kinds of nodes: clients, application servers, and database servers.

» An application server behaves as a client when accessing a database server

—A further extension is the **Peer-to-Peer** pattern. This is a distributed system architecture composed of various software components.

- Each of these components can be both a client and a server
- Any two components (peers) can set up a communication channel

A peer-to-peer architecture



A central server may be used to establish connections between peers

The Broker architectural pattern

- **The idea of the Broker architectural pattern is to distribute aspects of the software system *transparently* to different nodes.**
  - —Using the Broker architecture, an object can call methods of another object without knowing that this object is remotely located.
    - The Proxy design pattern can help to achieve this goal.
    - A Broker component (which handles all local invocations of objects) can be attached to each computer running distributed objects.
  - —CORBA (Common Object Request Broker Architecture) is a well-known open standard that allows you to build this kind of architecture.

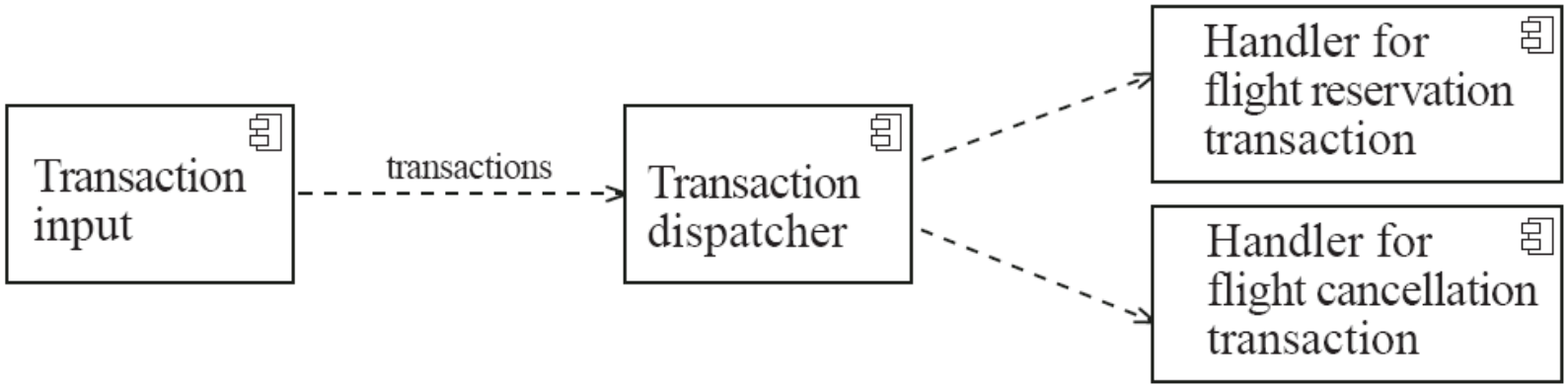
Example of a Broker system

The Transaction-Processing architectural pattern

In the Transaction-Processing architectural pattern:

- A process reads a series of inputs one by one.
  - —Each input describes a *transaction* – a command that typically makes some change to the data stored by the system.
- There is a transaction *dispatcher* component that decides what to do with each transaction.
- This dispatches a procedure call or message to one of a series of component (*transaction handlers*) that will *handle* the transaction.
  - —In each case, it is essential that the whole transaction be fully completed, or else not done at all.

Example of a transaction-processing system

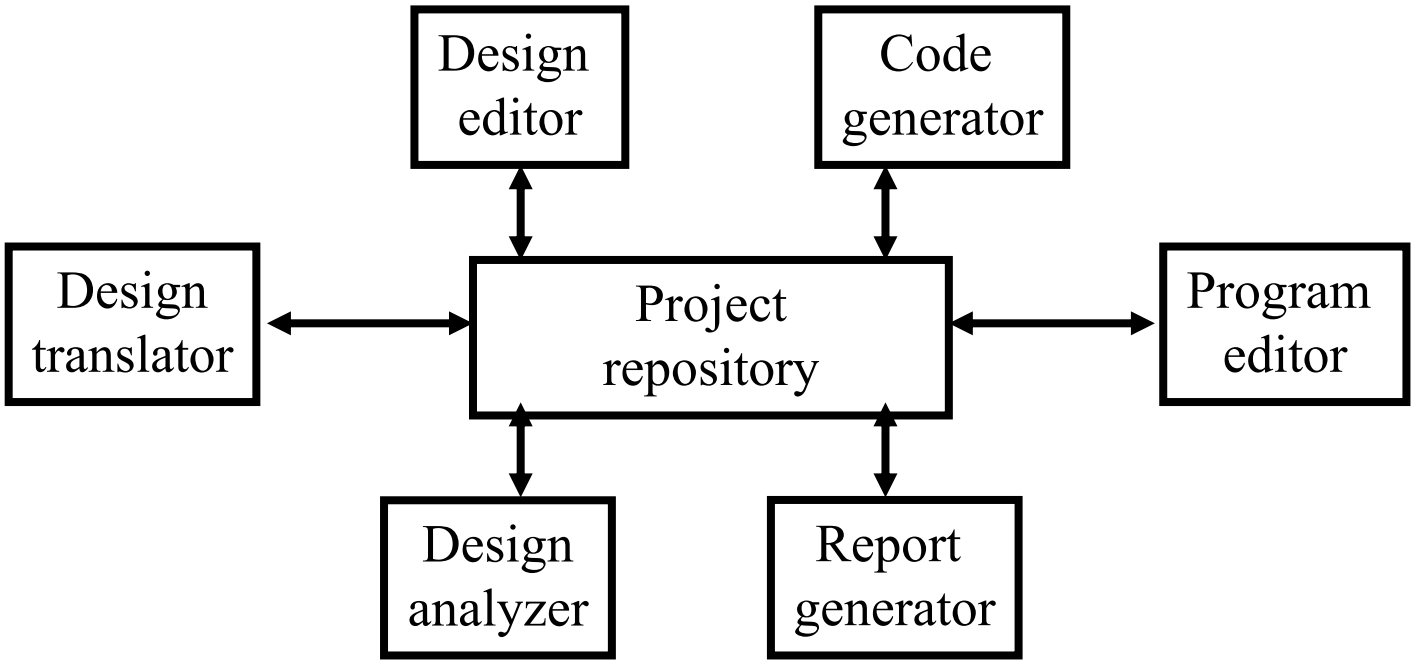


The transaction processing architecture used in the Airline system

The Repository architectural pattern

- **The repository model is a system model based on a database (or repository) shared by all sub-systems.**

  -Used e.g. in CASE toolsets design [Som04]:

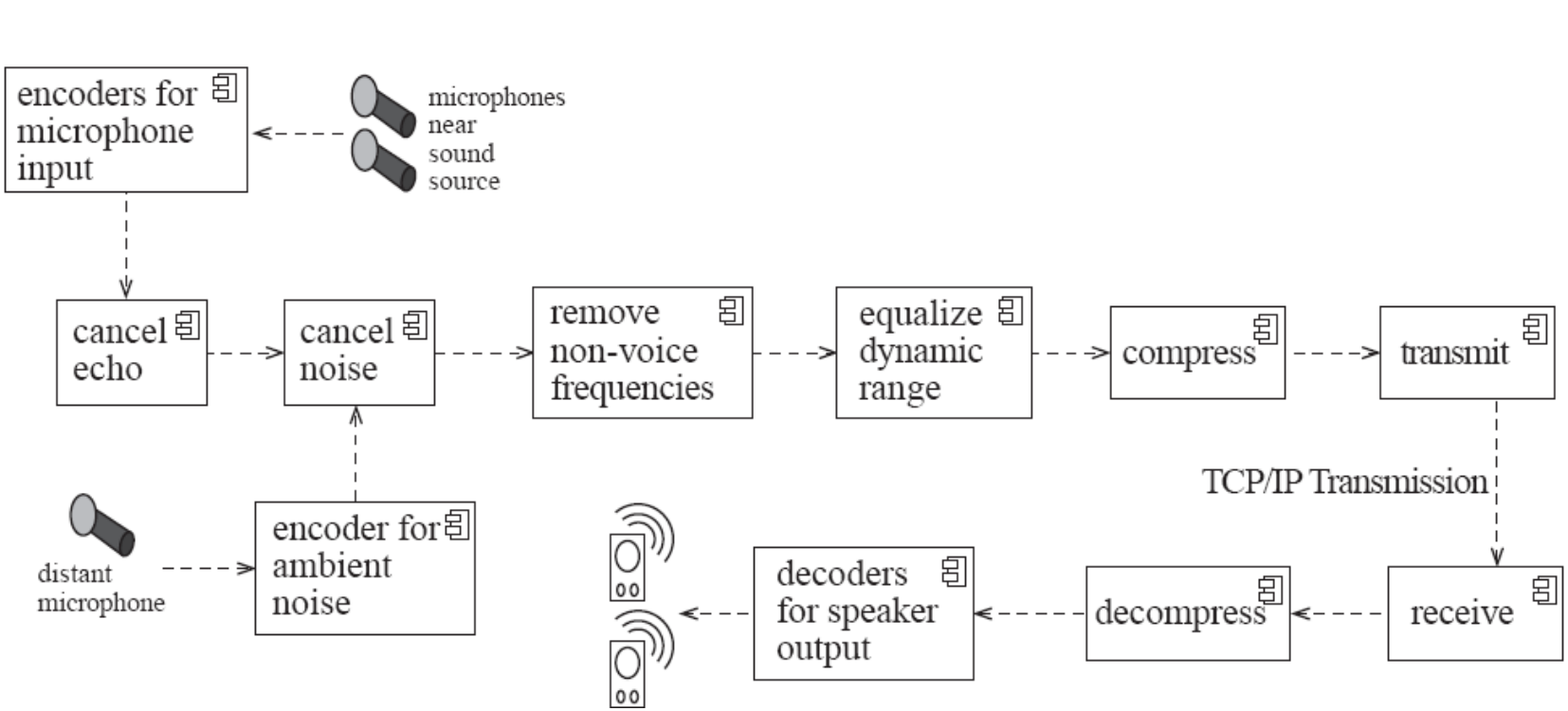


The Pipe-and-Filter architectural pattern

A stream of data, in a relatively simple format, is passed through a series of processes:

- Each of which transforms it in some way.
- Data is constantly fed into the pipeline.
- (Conceptually) The processes work concurrently.
- The architecture is very flexible.
  - —Almost all the components could be removed.
  - —Components could be replaced.
  - —New components could be inserted.
  - —Certain components could be reordered.

Example of a pipe-and-filter system



A Pipe-and-Filter architecture for sound processing

9.7 Writing a Good Design Document

Design documents as an aid to making better designs

- They force you to be explicit and consider the important issues before starting implementation.
- They allow a group of people to review the design and therefore to improve it.

Writing a Good Design Document

Design documents as a means of communication.

- To those who will be *implementing* the design.
- To those who will need, in the future, to *modify* the design.
- To those who need to create systems or subsystems that *interface* with the system being designed.

Structure of a design document

A. Purpose:

—What system or part of the system this design document describes.

—Make reference to the requirements that are being implemented by this design (*traceability)* .

B. General priorities:

—Describe the priorities used to guide the design process.

C. Outline of the design:

—Give a high-level description of the design that allows the reader to quickly get a general feeling for it.

D. Major design issues:

—Discuss the important issues that had to be resolved.

—Give the possible alternatives that were considered, the final decision and the rationale for the decision.

E. Other details of the design:

—Give any other details the reader may want to know that have not yet been mentioned.

When writing the document

- Avoid documenting information that would be *readily obvious* to a skilled programmer or designer.
- Avoid writing details in a design document that would be better placed as *comments* in the code.
- Avoid writing details that can be *extracted automatically* from the code, such as the list of public methods.

Additional references

[BRJ99] G. Booch, J. Rumbaugh and I. Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley, 1999.

[JBR99] I. Jacobson, G. Booch and J. Rumbaugh. *The Unified Software Development Process*. Addison-Wesley, 1999.

[Pre97] R. Pressman. *Software Engineering: A Practitioner's Approach*, (4th edition). McGraw-Hill, 1997.

[SG96] M. Shaw and D. Garlan. *Software Architecture*. Prentice-Hall, 1996.

[Som04] I. Sommerville. *Software Engineering,* (7th edition). Addison-Wesley, 2004.